VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Flood Situation Monitoring and Reporting System

By
Do Nguyen Chuong - ITITIU22021

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Computer Science

Ho Chi Minh City, Vietnam
2026

Flood Situation Monitoring and Reporting System

APPROVED BY:

Le Duy Tan, Ph.D., supervisor

THESIS COMMITTEE

Le Duy Tan, Ph.D

.....

Acknowledgments

I would also like to thank my thesis supervisor Dr. Le Duy Tan for his guidance and encouragement throughout the completion of this project. His comments helped me keep the conversation more on track, connect system design and implementation, and stay on track when the scope of the project threatened to get out of hand. I am particularly grateful to him for his time in reviewing my ideas and indicating areas that needed further clarification in my study.

Also, I would like to thank the researchers and lecturers of the Department of Computer Science and Engineering. The courses, discussions and academic support from the faculty gave me the foundational knowledge and technical skills required to develop VietFlood including support services, mobile applications, web interfaces and infrastructure deployment.

Firstly I would like to thank my family for their continued patience and support during my studies. Their trust in me gave me the motivation and the space to finish this thesis. I am also indebted to the friends and colleagues who encouraged me along the way. Some were short but got me through tough times.

I wish to thank sincerely the many people who have been generous, helpful and supportive in the course of this work.

Table of Contents

List of Tables

List of Figures

List of Abbreviations

API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Deployment
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
HTTP	Hypertext Transfer Protocol
UI/UX	User Interface / User Experience
CDN	Content Delivery Network
PUB/SUB	Publish–Subscribe
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
JWT	JSON Web Token
TTL	Time To Live
WS	WebSocket

Abstract

Flood response depends on small pieces of local information becoming useful at the right time. In many cases, the first sign of a flood problem is not an official record but a photo, a short description, or a shared location from a person already near the affected area. When those signals stay inside separate chat groups or informal channels, the information is hard to verify, search, and follow up. This thesis presents VietFlood, a prototype for a Flood Situation Monitoring and Reporting System that organizes citizen flood reports for relief awareness and administrator review.

VietFlood is implemented as a multi-platform prototype. The backend uses NestJS with an API gateway, an authentication service, a reports service, and a Socket.IO tracking gateway. RabbitMQ is used for service-to-service communication. PostgreSQL is used to store users, refresh tokens and flood reports using TypeORM. Redis is used to support cached report lookup. Cloudinary is used to store uploaded evidence files. The common infrastructure code is under `vietflood_common` with reusable logging, Redis and Cloudinary utilities. The mobile app is designed with Expo React Native for citizen and relief workflows. The online dashboard is designed with Next.js, React, Typescript and Tailwind CSS for the admins. The prototype demonstrates the key workflow from report creation to review. Citizens can register, login, create Flood Report, offer administration address fields and coordinates, urgency and severity, submit photo evidence. Relief users can explore wider report views and map related screens. The web dashboard enables administrators to access reporter information, search and filter reports, preview evidence, manage users and alter report state utilizing pending, verified, resolved and rejected statuses. VietFlood additionally has live tracking path where linked clients transmit location updates over Socket.IO after latitude and longitude confirmation at the gate. The work focuses on system design and implementation, not flood prediction or emergency dispatch. The final prototype illustrates the integration of mobile reporting, role-based access control, evidence storage, real-time location events, shared backend utilities, and container-based deployment in a flood reporting system that is specifically designed for Vietnam. While it is only a prototype, VietFlood gives a functioning basis for the collection of field reports and makes them easier to review. Keywords: VietFlood, flood reporting, relief coordination, real-time tracking, microservice design, NestJS, React Native, Next.js, RabbitMQ, PostgreSQL, Redis, Cloudinary

Chapter 1
Introduction

This chapter gives the inspiration for VietFlood and defines the problem solved by the thesis. Flood events create immediate, local information: images of flooded roadways, text messages on water levels, and location pins sent among neighbors and helpers. These signals can be beneficial; however, they are challenging to verify and repurpose when they are isolated in conversation messages. VietFlood is meant to turn these observations into structured reports that can be kept, searched, evaluated and tracked over time. This connects with the bigger idea of leveraging citizen-generated crisis information as an input for disaster response if that information is structured into usable data [1, 2].

1.1 Motivation and Problem Definition

In genuine flood scenarios, the first important information is usually from the field, not from official sources. However, field reports are often inconsistent and fragmentary. A message can have a photo, but not an administrative address. You can share a location pin with no explanation. Reports may be duplicates and describe the same incident in different words. When information is dispersed in this way, it becomes difficult for a reviewer to answer simple questions consistently: what happened, where it happened, who reported it, and if the case has been validated or handled.

The thesis challenge might be described as: *how to develop a workable reporting system that converts ad-hoc flood observations into structured, reviewable records, supporting diverse user roles, and being simple enough to execute as a prototype.* VietFlood aims to solve this challenge by developing a multi-platform prototype that enables report submission from mobile devices, uploading evidence, persistent storage, role-based access control, administrator review and status monitoring.

1.2 Objectives of the Research

The primary aim is to design and build a functional prototype for reporting and reviewing flood situations. The prototype shall demonstrate the end-to-end workflow, not just a user interface or just a backend.

Some aims are:

1. Develop a reporting data model including location fields, evidence information, timestamp, and a review status suitable for follow-up by an administrator.

2. Implement a backend that divides public API handling, authentication, and report management responsibilities via a minor service split [3, 4, 5].

3. Implement JWT Authentication with role-based access control to provide varied capabilities to citizens, relief users and administrators [6].

4. Persistently store reports and user accounts using PostgreSQL and TypeORM and store evidence metadata as structured JSON when appropriate [7, 8].

5. Store photo evidence externally to the database using a media platform and retain references in the report record [9].

6. Provide a citizen reporting and relief-oriented browsing mobile application utilizing Expo React Native [10, 11].

7. Provide an administrator online dashboard with Next.js to evaluate reports, inspect evidence, and update report status [12].

8. Package the system to run in a repeatable environment using Docker Compose and be deployable using CI/CD practices [13, 14, 15, 16, 17].

VietFlood is a prototype for collecting and reviewing information. It does not offer flood prediction, hydrological modelling, or automated dispatch. The prototype presupposes that users can access the mobile application and that evidence uploads may be feasible; however, it also recognizes that connectivity may be problematic during floods. The system therefore depends on an explicit data model, a role-based review procedure, and an architecture that can be tested locally and modified in the future.

1.3 System Description

There are three front-end parts and a backend to VietFlood:

- **Mobile app:** a citizen and relief client created with Expo React Native. Citizens can check, file reports with location and optional evidence and view personal report history. Relief users can get more report information and map-related views [10, 11].

- **Web dashboard:** administrator interface designed using Next.js to examine reports, inspect evidence, filter and update status [12].

- **Backend services:** a NestJS-based backend exposing one public API surface via an API gateway and internal services for authentication and report management [18, 19].

- **Supporting infrastructure:** RabbitMQ for service messaging, Redis for caching, PostgreSQL for persistence, and Cloudinary for evidence storage [20, 21, 8, 9].

The essential design decision is that a report is considered to be a durable record. It's more than a message. Reports have a description, location fields and coordinates, references to proof, and a status that can be amended by the administrators after evaluation.

This thesis contributes as follows:

- A straightforward, Vietnam-centric reporting process that distinguishes between citizen input, relief browsing and administrator validation as independent functions.

- A fully functional multi-platform prototype (mobile + web + backend) demonstrating the complete report production process, including evidence upload, permanent storage, and status tracking.

- A service-oriented backend structure that separates authentication and reporting tasks while staying small enough for a thesis prototype.

- An API surface and assessment checklist, including Appendix material containing a set of endpoints, response samples, and scenario-based test cases.

1.4 Chapter Roadmap

The structure of the thesis is as follows. In Chapter 2 background and related work on crisis information, structured reporting and the architectural patterns employed by VietFlood are introduced. Chapter 3 describes the methodology and design process covering actor modeling, system structure and data model options. Chapter 4 describes the prototype implementation in backend services, the mobile application and the administrator dashboard. Chapter 5 reviews the prototype with walkthroughs of workflows and discussion of limitations. The last chapter, Chapter 6, is the thesis conclusion and future enhancements.

Chapter 2
Background and Related Works

2.1 Flood Reporting as an Information Problem

Flood reporting is an information problem before it is a software problem. The earliest signals of a flooded road or a blocked route usually come from people already near the affected area. These field observations arrive quickly, but they arrive in formats that are hard to reuse: a photo without a clear address, a short message without coordinates, or a location pin without enough context to judge severity. As a result, valuable information can be lost in message streams or become difficult to verify and track over time.

VietFlood follows the idea that citizen-generated information can support crisis response when it is organized into usable data. Research on crowdsourcing and crisis information highlights that local observations can provide timely coverage, but they require structure and processing to become useful in practice [1, 2]. In this thesis, a "report" is treated as a durable record with fields that support review: what happened, where it happened, who reported it, what evidence exists, and what the current status is.

2.1.1 Review States and Follow-Up

The simplest trust signal in a reporting platform is the review status. VietFlood uses a small status set suitable for a prototype: `pending`, `verified`, `resolved`, and `rejected`. A new submission is pending. An administrator can mark it verified after review, resolved when it is no longer active, or rejected when the report is incorrect or unsuitable. Keeping rejected reports separate supports accountability by preserving the record of what was submitted rather than silently discarding it.

Figure 2.1 summarizes the report lifecycle used in the prototype. The purpose of the lifecycle is not to model every possible response action. Its purpose is to ensure that reports remain visible and reviewable after the initial submission, with a status that can be updated as more information becomes available.

2.1.2 Mobile Reporting Constraints

Reporting is expected to happen in the field, so mobile interaction is central. A mobile-first flow prioritizes short forms, simple location entry, and optional evidence upload rather than requiring long text input. VietFlood uses Expo React Native for the citizen and relief clients [10, 11]. To reduce inconsistent spelling of administrative locations, the system uses the Vietnam Provinces Open API as a structured source for provinces and wards [22]. This choice improves filterability and reduces ambiguity when multiple users describe the same area differently.

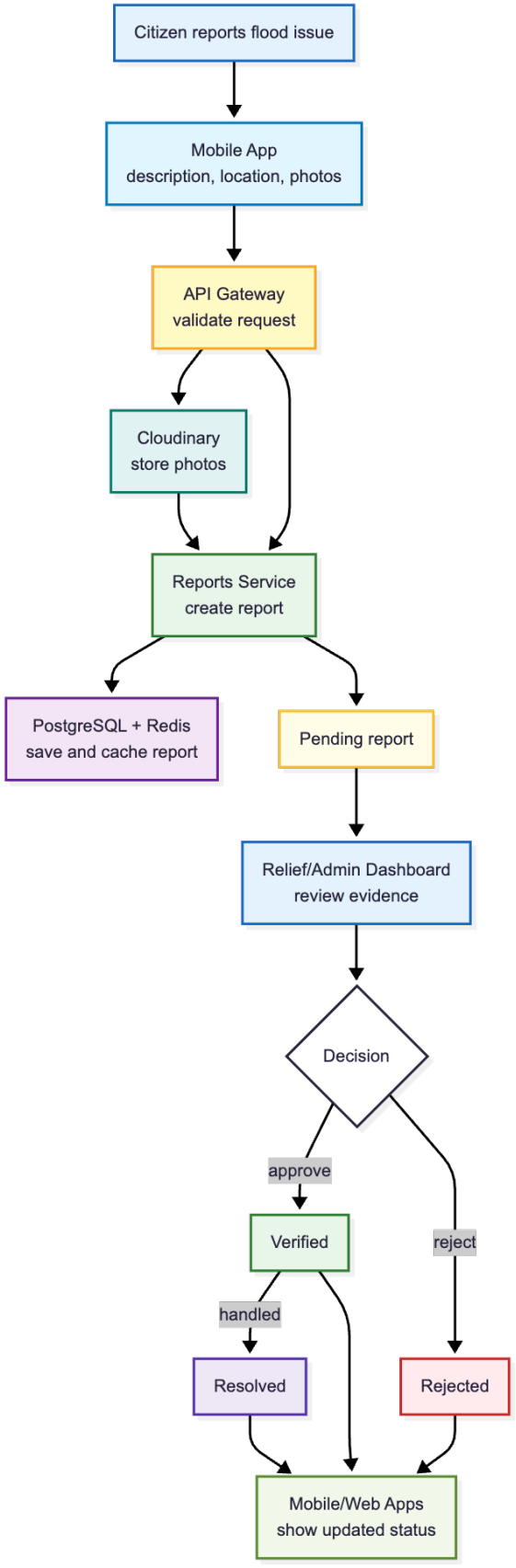


Figure 2.1: VietFlood report flow from mobile submission to administrator review status.

2.2 Research Context and Key Concepts

2.2.1 Citizen Signals in Emergencies

Two streams of related work motivate VietFlood. The first is crowdsourcing and volunteered geographic information in disasters. Goodchild and Glennon describe crowdsourcing geographic information as a research frontier in disaster response, highlighting both its potential value and the need for methods that transform observations into usable information products [1]. The second is crisis communication through social media and messaging. Imran et al. survey methods for processing emergency-related social media messages and emphasize the challenges of noise, duplication, and incomplete context [2]. VietFlood does not attempt automated social media extraction, but it is designed to address the same practical problem: field information arrives quickly and inconsistently, and needs structure to support review.

2.2.2 Responsibility-Driven Architecture

The background for VietFlood's implementation is software architecture that makes responsibilities explicit. Architectural guidance emphasizes designing system structure around responsibilities and quality attributes, rather than only selecting technologies [23]. For this thesis, the most important responsibilities are: (1) public API handling and access control, (2) authentication and session management, (3) report persistence and status updates, and (4) evidence upload and retrieval. The design should also remain operable as a small prototype without a complex production platform.

2.3 Backend Decomposition Options

A single monolithic backend can be an effective approach for small projects. However, as features accumulate, authentication, file upload handling, report persistence, caching, and real-time communication can become tightly coupled. Microservice architecture is often defined as decomposing a system into services around capabilities, trading increased operational complexity for clearer boundaries and independent evolution [3, 4, 5].

VietFlood adopts a modest service split suitable for a thesis prototype. The system uses an API gateway as the client-facing entry point, and internal services for authentication and report management. Supporting infrastructure (RabbitMQ, Redis, PostgreSQL, Cloudinary) is introduced only when it directly supports a needed responsibility.

2.3.1 Gateway Layer and Service Messaging

The API gateway pattern provides one stable surface for clients even when the backend is split internally. In VietFlood, clients call the gateway for authentication and report routes. The gateway then forwards work to internal services using NestJS microservice patterns [18, 19]. This method reduces coupling between clients and internal service topology.

Service-to-service communication uses RabbitMQ request–response messaging [20]. Using explicit messages makes boundaries visible: authentication logic lives in the auth service, while report storage and status changes live in the reports service. The gateway coordinates requests, enforces access control, and handles evidence upload, but it does not directly own all business logic.

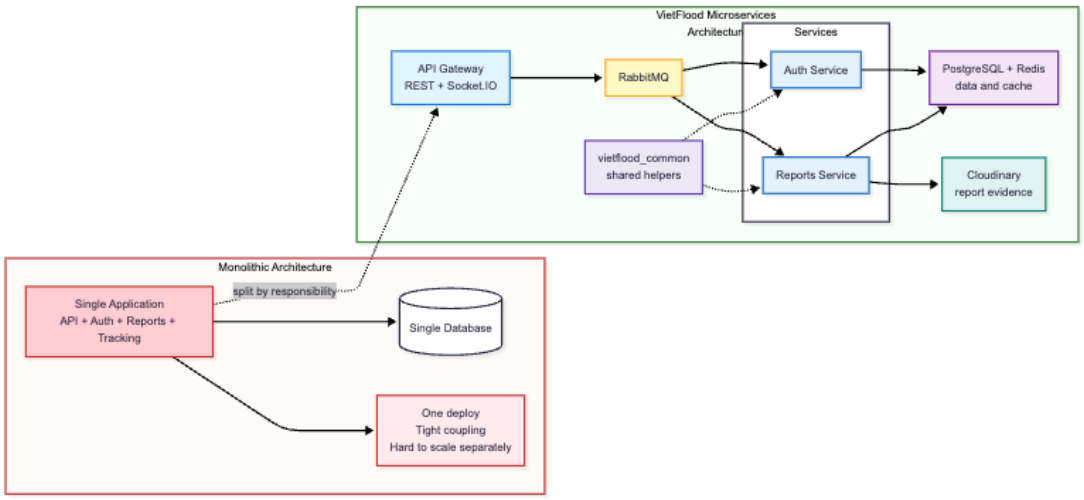


Figure 2.2: High-level contrast between a monolithic backend and the VietFlood service split.

2.4 Storage, Evidence, and Caching Choices

2.4.1 Relational Storage and JSON Evidence Metadata

VietFlood uses PostgreSQL to store users, reports, and evidence metadata. TypeORM maps application entities to relational tables [7]. Evidence items are stored as structured JSON metadata (for example, Cloudinary URL and public ID). PostgreSQL's JSON support allows this flexible structure while still using a relational database for core report fields such as status and coordinates [8].

2.4.2 External Evidence Storage (Cloudinary)

Storing images as database blobs is not necessary for the thesis prototype and complicates storage and delivery. VietFlood uploads evidence to Cloudinary and stores only references and metadata in the report record [9]. This enables evidence preview in the administrator dashboard while keeping report records small enough to query and update efficiently.

2.4.3 Redis Cache Layer

Redis is used as a cache layer to reduce repeated reads for frequently requested report details [21]. Caching is kept conservative in the prototype. PostgreSQL remains the source of truth, while Redis stores temporary copies that can be refreshed after report updates. This design avoids complex cache invalidation rules while still demonstrating a realistic backend optimization approach.

2.5 Access Control and Real-Time Channels

2.5.1 JWT Tokens and Password Hashing

Role-based access control requires reliable authentication. VietFlood uses bcrypt hashing for passwords [24] and JWT access tokens for stateless verification of identity and role claims [6]. Refresh tokens are used to maintain longer sessions while allowing access tokens to remain short-lived.

Figure 2.3: Example views of evidence upload on mobile and evidence review on the dashboard.

2.5.2 Socket.IO for Live Location Updates

Real-time communication can support situational awareness when users share location updates. VietFlood includes a Socket.IO gateway for live location broadcasting [25]. The prototype validates coordinate ranges and broadcasts events between connected clients. This provides a minimal foundation for live tracking without expanding scope into persistent tracking histories or assignment workflows.

2.6 Deployment and Environment Repeatability

Repeatable environments are important even for prototypes. VietFlood uses Docker Compose to run backend services and infrastructure dependencies locally [13]. The deployment approach follows standard CI/CD practices: images can be built and deployed automatically using GitHub Actions and Azure App Service container deployment [14, 15]. Continuous delivery guidance motivates keeping build and deployment steps repeatable and automated rather than relying on manual server setup [16, 17].

Chapter 3
Methodology

This chapter gives the inspiration for VietFlood and defines the problem solved by the thesis. Flood events create immediate, local information: images of flooded roadways, text messages on water levels, and location pins sent among neighbors and helpers. These signals can be beneficial; however, they are challenging to verify and repurpose when they are isolated in conversation messages. VietFlood is meant to turn these observations into structured reports that can be kept, searched, evaluated and tracked over time. This connects with the bigger idea of leveraging citizen-generated crisis information as an input for disaster response if that information is structured into usable data [1, 2].

3.1 Motivation and Problem Definition

In genuine flood scenarios, the first important information is usually from the field, not from official sources. However, field reports are often inconsistent and fragmentary. A message can have a photo, but not an administrative address. You can share a location pin with no explanation. Reports may be duplicates and describe the same incident in different words. When information is dispersed in this way, it becomes difficult for a reviewer to answer simple questions consistently: what happened, where it happened, who reported it, and if the case has been validated or handled.

The thesis challenge might be described as: *how to develop a workable reporting system that converts ad-hoc flood observations into structured, reviewable records, supporting diverse user roles, and being simple enough to execute as a prototype.* VietFlood aims to solve this challenge by developing a multi-platform prototype that enables report submission from mobile devices, uploading evidence, persistent storage, role-based access control, administrator review and status monitoring.

3.2 Objectives of the Research

The primary aim is to design and build a functional prototype for reporting and reviewing flood situations. The prototype shall demonstrate the end-to-end workflow, not just a user interface or just a backend.

Some aims are:

1. Develop a reporting data model including location fields, evidence information, timestamp, and a review status suitable for follow-up by an administrator.

2. Implement a backend that divides public API handling, authentication, and report management responsibilities via a minor service split [3, 4, 5].

3. Implement JWT Authentication with role-based access control to provide varied capabilities to citizens, relief users and administrators [6].

4. Persistently store reports and user accounts using PostgreSQL and TypeORM and store evidence metadata as structured JSON when appropriate [7, 8].

5. Store photo evidence externally to the database using a media platform and retain references in the report record [9].

6. Provide a citizen reporting and relief-oriented browsing mobile application utilizing Expo React Native [10, 11].

7. Provide an administrator online dashboard with Next.js to evaluate reports, inspect evidence, and update report status [12].

8. Package the system to run in a repeatable environment using Docker Compose and be deployable using CI/CD practices [13, 14, 15, 16, 17].

VietFlood is a prototype for collecting and reviewing information. It does not offer flood prediction, hydrological modelling, or automated dispatch. The prototype presupposes that users can access the mobile application and that evidence uploads may be feasible; however, it also recognizes that connectivity may be problematic during floods. The system therefore relies on an explicit data model, a role-based review procedure, and an architecture that can be tested locally and extended in the future.

3.3 System Description

There are three front-end parts and a backend to VietFlood:

- **Mobile app:** a citizen and relief client created with Expo React Native. Citizens can check, file reports with location and optional evidence and view personal report history. Relief users can get more report information and map-related views [10, 11].

- **Web dashboard:** administrator interface designed using Next.js to examine reports, inspect evidence, filter and update status [12].

- **Backend services:** a NestJS-based backend exposing one public API surface via an API gateway and internal services for authentication and report management [18, 19].

- **Supporting infrastructure:** RabbitMQ for service messaging, Redis for caching, PostgreSQL for persistence, and Cloudinary for evidence storage [20, 21, 8, 9].

The essential design decision is that a report is considered to be a durable record. It's more than a message. Reports have a description, location fields and coordinates, references to proof, and a status that can be amended by the administrators after evaluation.

This thesis contributes as follows:

- A straightforward, Vietnam-centric reporting process that distinguishes between citizen input, relief browsing and administrator validation as independent functions.

- A fully functional multi-platform prototype (mobile + web + backend) demonstrating the complete report production process, including evidence upload, permanent storage, and status tracking.

- A service-oriented backend structure that separates authentication and reporting tasks while staying small enough for a thesis prototype.

- An API surface and assessment checklist, including Appendix material containing a set of endpoints, response samples, and scenario-based test cases.

3.4 Chapter Roadmap

The structure of the thesis is as follows. In Chapter 2 background and related work on crisis information, structured reporting and the architectural patterns employed by VietFlood are introduced. Chapter 3 describes the methodology and design process covering actor modeling, system structure and data model options. Chapter 4 describes the prototype implementation in backend services, the mobile application and the administrator dashboard. Chapter 5 reviews the prototype with walkthroughs of workflows and discussion of limitations. The last chapter, Chapter 6, is the thesis conclusion and future enhancements.

Chapter 4
Implementation and Result

This chapter discusses the execution of the VietFlood prototype, as well as the concrete deliverables produced by the project. The system was put in place to offer a complete reporting loop: users authenticate, citizens submit a report with location and optional proof, the backend stores and serves that report via a stable API, and administrators examine and update report status.

The implementation comprises four main artifacts: 1) a backend consisting of an API gateway, an authentication service and a reports service; 2) a mobile application for citizen and relief users; 3) a web dashboard for administrators; and 4) a shared library, `vietflood_common` providing reusable infrastructure across services.

4.1 Prototype Deliverables and Tech Stack

For consistency in models, validation rules, and data contracts while development, TypeScript is used in both the server and the client apps. The backend services are developed using NestJS [18] and the NestJS microservice support is used to interact over a message broker [19]. The mobile application is constructed using Expo React Native [10, 11] and the web dashboard is built using Next.js [12].

The prototype is designed to run in containers. For local development, Docker Compose is used to launch the gateway, services, RabbitMQ, and Redis in a repeatable environment [13]. For deployment, container images are deployed to Azure App Service [15] and build/deploy automation is done with GitHub Actions [14]. Sensitive parameters such as the database URL, JWT secrets, Cloudinary credentials, and broker credentials are passed in using environment variables and not saved in source code.

Table 4.1: Technologies employed in the VietFlood prototype.

Component	**Implementation choices**
Backend	NestJS services, TypeORM persistence, RabbitMQ communication, Redis cache, and JWT authentication.
Shared library	`vietflood_common` for logging, Redis helpers, and Cloudinary upload utilities.
Evidence storage	Cloudinary for media uploads and public delivery URLs.
Mobile app	Expo React Native, secure token storage, image picker, and location services.
Web dashboard	Next.js, role-based routes, report review UI, and admin status updates.
Deployment	Docker Compose for local runtime, GitHub Actions automation, and containers on Azure App Service.

4.2 Implementation of the Backend

4.2.1 Responsibilities for Routing and Service

API gateways are the sole components of the infrastructure that are accessible to clients. It handles HTTP requests, including multipart uploads, verifies access using guards, and dispatches internal work to the authentication or reporting service. Internal communication uses RabbitMQ request–response message patterns [20, 19]. It keeps transport and access control together, and moves business logic and database operations into separate services.

Table 4.2: Distribution of responsibility among backend services in VietFlood.

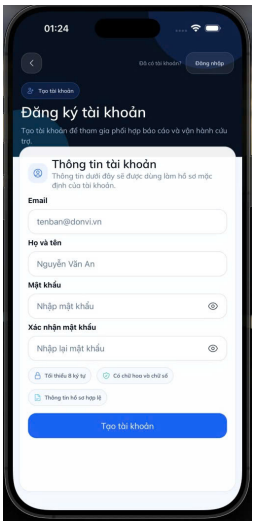
Service	**Main responsibilities**
API gateway	Public REST routes, file parsing for uploads, JWT and role guards, and routing requests to internal services.
Auth service	User registration, password validation, token issuance, refresh token rotation, and user administration.
Reports service	Report persistence, evidence metadata storage, report listing and filtering, status updates, and cache invalidation.

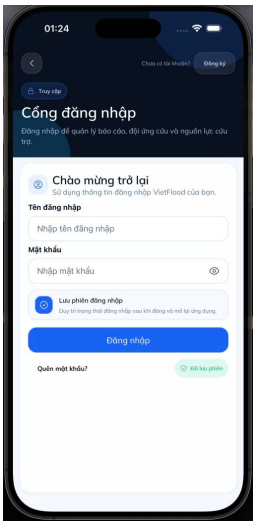
Figure 4.1: Message routing from the API gateway to internal services through RabbitMQ.

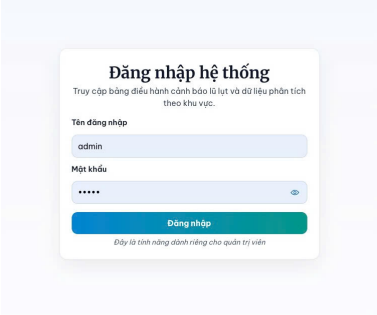
4.2.2 Authentication, Tokens and Role Checks

VietFlood employs password authentication. We hash passwords with bcrypt before storage [24]. After login the authentication service returns an access token and refresh token. Access tokens are JSON Web Tokens that hold identity and role claims needed by the gateway authorization guards [6]. Refresh tokens are saved and rotated to enable longer-lived sessions and to reduce the impact of a token leak.

The prototype has three supported roles: `citizen`, `relief` and `admin`. Protected endpoints and admin-only routes have role checks at gateway. This is a simplistic model, but it is sufficient to illustrate the distinct responsibilities: residents submit reports, relief users browse and watch, and administrators assess and alter status.







Citizen registration view. Mobile sign-in view. Administrator sign-in view.

Figure 4.2: Authentication screens for citizen registration, mobile login, and administrator login.

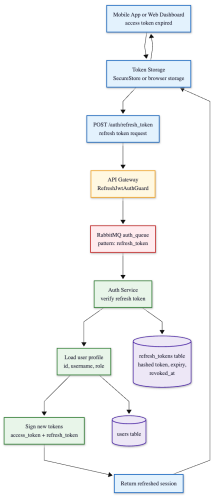


Figure 4.3: Refresh token rotation sequence to get new access tokens and invalidate old refresh tokens.

4.2.3 Evidence Pipeline and Report Handling

Flood reports are produced from the mobile client and received by the gateway as multipart form data. In the presence of evidence photos, the gateway uploads files to Cloudinary and creates a list of metadata, including delivery URL, public ID, and resource type [9]. The final payload is subsequently transmitted to the reports service via RabbitMQ. The reports service validates the payload, persists the report using TypeORM [7], and writes a cache item in Redis for speedier subsequent lookups [21].

This evidence-first pipeline keeps the database focused on structured report fields but still makes evidence available for evaluation. It also makes the report contents self-contained; administrators can review a report by reading one database row plus associated evidence information, without requiring local file storage.

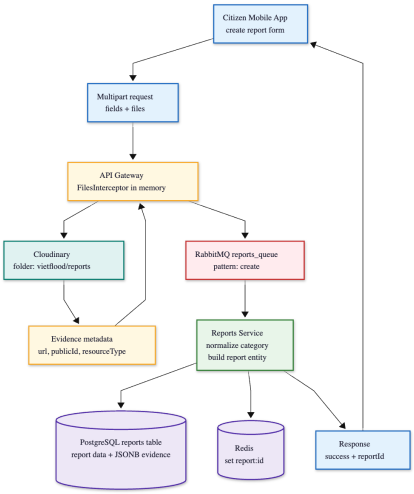


Figure 4.4: End-to-end report processing pipeline from submission to persistence and cache refresh.

Update and remove actions are subject to the same service boundary. The gateway enforces role permissions and passes update/delete orders to the reports service. The reports service invalidates and updates cache entries when a report changes so that subsequent reads match the current state.

4.2.4 Utilities for Shared Infrastructure

In order to keep services minimal and consistent, VietFlood contains a shared library, `vietflood_common`, exposing a logging module and modest wrappers for Redis and Cloudinary. Centralization of these utilities minimizes duplicated code and facilitates consistent error handling and configuration across services.

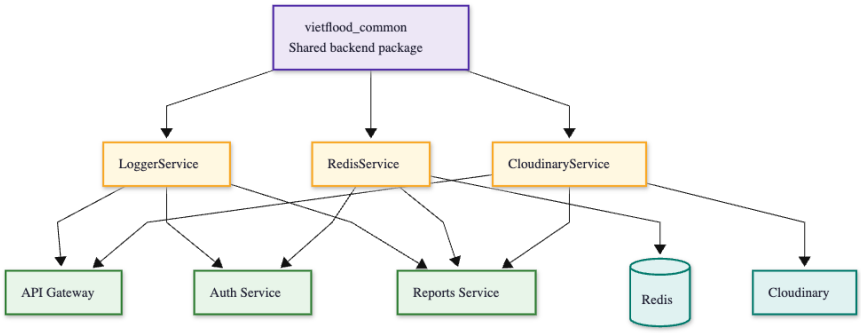


Figure 4.5: Shared utility package used by backend services for logging, Redis, and Cloudinary.

4.2.5 Live Location Event Channel

The prototype uses a Socket.IO gateway for live location broadcasting [25]. Clients submit `send-location` events with lat and long values. The gateway checks the coordinate ranges and publishes valid updates to other connected clients using a `receive-location` event. Invalid payloads cause a `location-error` response to the sender. This feature displays event flow as it occurs. It can be enhanced with authorized identities and persistence if needed for operational use.

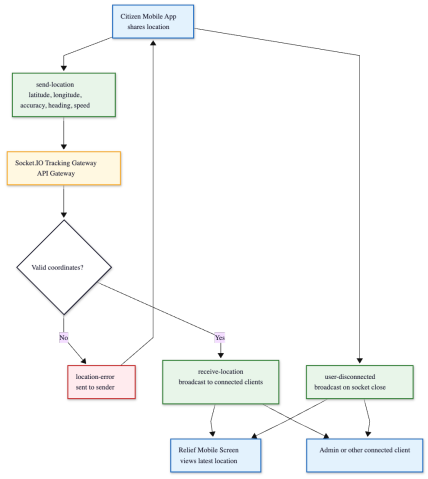


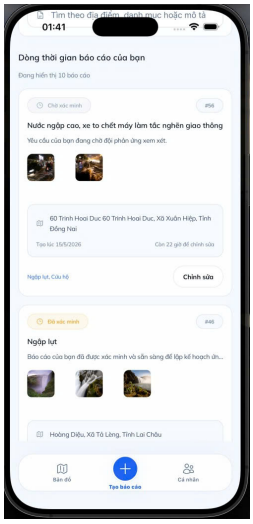
Figure 4.6: Socket.IO location event flow including validation, broadcast, and error response.

4.3 Client Side Implementation

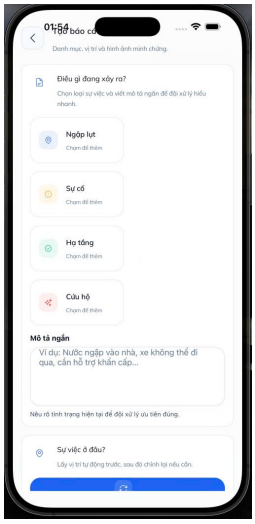
4.3.1 Mobile App Capabilities

The mobile application provides the field-facing interface, and is designed for rapid submission and reading. Once signed in the app loads the current profile and shows a report list for the user. Citizens can fill in a category, a short description, administrative address fields and coordinates and generate a report. The make report request includes the selection and upload of evidence photographs from the device.

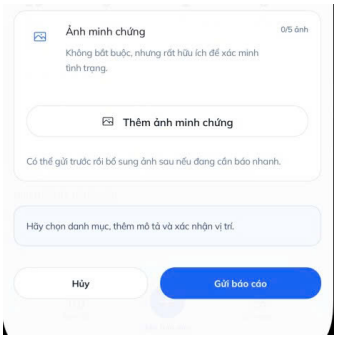
The relief role is related to map and browsing views. Relief users can view report listings and inspect report details, although they do not modify report status in this prototype. That separation prevents competing status updates and keeps the verification decision inside the administrator workflow.



Citizen report list.

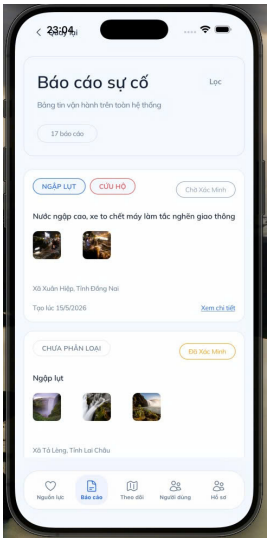


Report creation form.

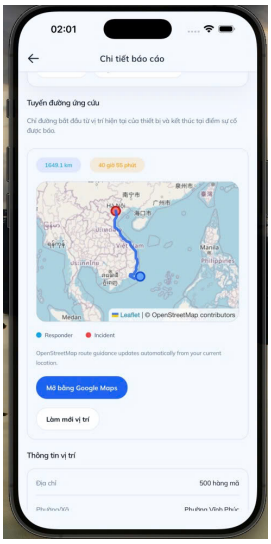


Evidence upload workflow.

Figure 4.7: Mobile application screens for report listing, report creation, and evidence upload.



Relief list view.



Relief map view.

Figure 4.8: Relief user screens for browsing reports and viewing their spatial distribution.

4.3.2 Admin Panel Features

The online dashboard handles report review and user administration. This loads the current profile after login and prevents routes that require admin access. Administrators can view incoming reports, filter them by status, see evidence thumbnails and change status values such as `pending`, `verified`, `resolved` and `rejected`. Status changes are delivered to the gateway and then sent to the reports service over RabbitMQ to keep the database and cache coherent.

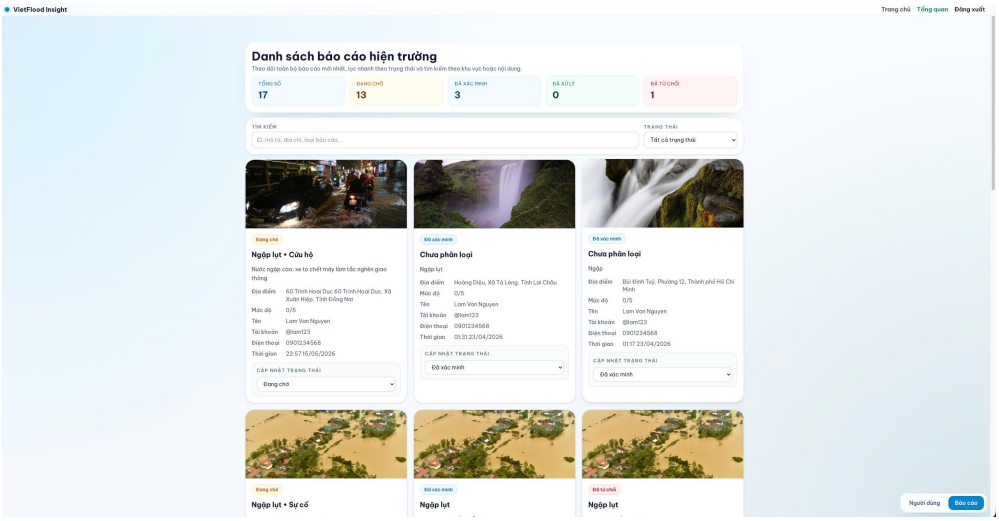


Figure 4.9: Administrator dashboard overview showing report cards, filters, and evidence thumbnails.

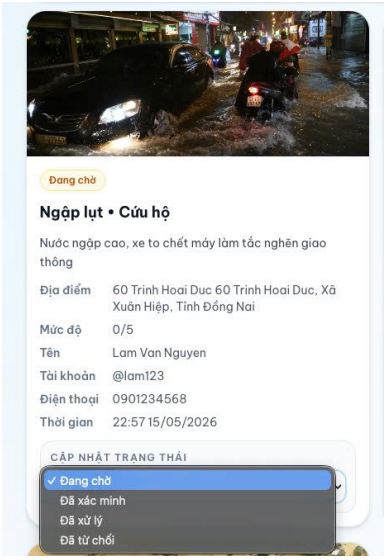


Figure 4.10: Evidence review and status update screen for administrators.

4.4 Automating Release and Deployment

The prototype runtime is containerized for consistency between local and deployed environment. For local development, Docker Compose is utilized to launch all dependencies, Redis and RabbitMQ, in addition to the backend services [13]. Images are created and published, and Azure App Service is set up to execute the most recent images using environment configuration supplied by the platform [15]. This is a straightforward container-based deployment method.

Automation is done with GitHub Actions to produce images, submit them to a registry and update the deployment configuration [14]. In line with continuous delivery approach, which maintains build and deploy logic in version-controlled pipelines [16, 17], this minimizes manual processes.

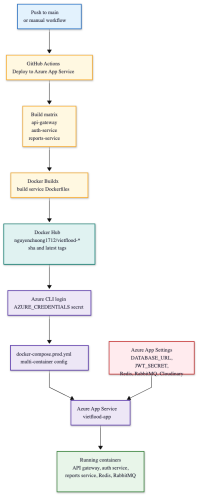


Figure 4.11: CI/CD pipeline from source push to container build, publish and Azure App Service deployment.

4.5 Summary of the Results

VietFlood can be launched locally with Docker Compose and exercised via the mobile and web clients at the final stage of implementation. The backend exposes REST endpoints for authentication and report administration via the API gateway. A complete list of endpoints is described in Appendix A, Table A.1, with representative response samples utilized in the evaluation.

In practice, the system exhibits the expected function split: residents submit reports, relief users explore location-aware views and administrators analyze and amend report status. These results are assessed in Chapter 5 using scenario walkthroughs and functional test checklists.

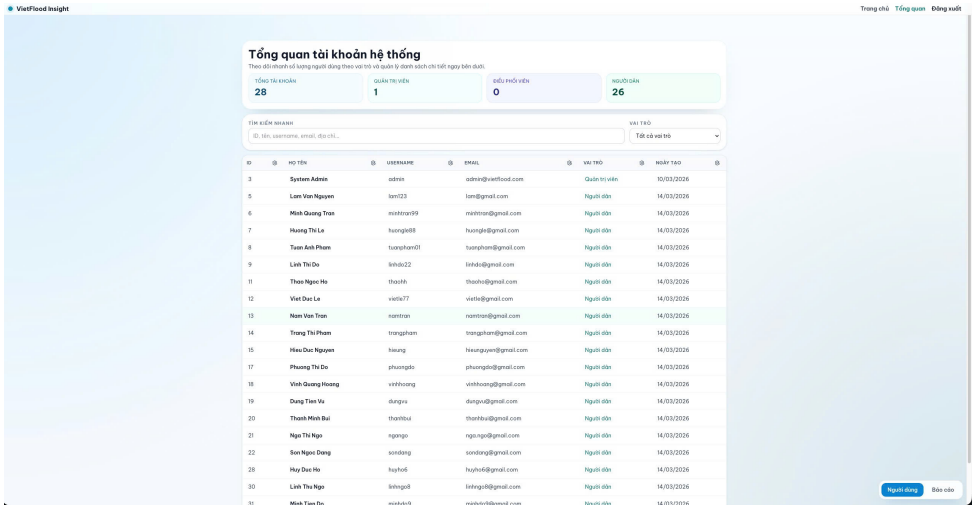


Figure 4.12: User management view to browse accounts and responsibilities in the dashboard.

Chapter 5
Evaluation

This chapter reviews the VietFlood prototype as an implemented flood reporting system. The evaluation is not intended to measure real-world impact in a live flood relief operation. Instead, it evaluates if the delivered backend services and client apps support the workflow described in Chapter 4: users may authenticate, submit reports with location and proof, save and retrieve reports and review reports through role-based interfaces.

The evaluation is conducted via scenario walkthroughs and source-level inspection of the API gateway, authentication service, reporting service, mobile application, online dashboard, and shared library. The full list of the REST endpoints and indicative replies may be found in Appendix A (Table A.1). The system-level test cases that are used as a checklist for this evaluation are shown in Appendix C (Tables C.1–C.3). In this instance, the results are more like a prototype validation than a certification for production.

5.1 Evaluation Methods

The evaluation is organized around three user roles offered by VietFlood: **Citizen:** sign up and sign in, submit flood reports with location and evidence, browse own report history on mobile. **Relief user:** navigate report information and location-based views on mobile. **Administrator:** examine reports, check evidences, update report status, and manage users on the web dashboard.

On the backend, the API gateway is the public interface for HTTP routes and Socket.IO events[25]. The auth service handles authentication and profile logic. Report generation, update, deletion and cache handling are the responsibility of the reports service. The gateway communicates with internal services using RabbitMQ message patterns [20]. The division is consistent with the objective of having clear service roles and maintaining the system manageable as a prototype [23].

5.2 Coverage of Workflows

The main functional need is the report procedure from start to finish. Citizen submits report through mobile client. The gateway handles multipart form data, uploads evidence files to Cloudinary [9] and passes the final payload to the reports service over RabbitMQ [20]. The reports service persists the report in PostgreSQL and stores evidence metadata as JSON fields [8]. After creation and update operations, redis cache item is written to allow later lookups [21].

On the review side, administrators utilize the web dashboard to list reports, check evidence thumbnails, and update status values such as `pending`, `verified`, `resolved` and `rejected`. Relief users can read report information through role limited endpoints and display it in list and map oriented views on the mobile application. Combined these features constitute the desired loop: submission, storage, review and update of status.

Table 5.1: Observed coverage of important VietFlood workflows in the prototype.

Workflow	**Primary interface**	**Observed behavior in prototype**
Citizen sign in and profile access	`/auth/sign_in`, `/auth/profile`	Tokens are issued after password validation, protected profile data is returned when a JWT is provided.
Create report with evidence	`/reports/create` (multipart)	Evidence files are uploaded to Cloudinary and stored as metadata; the report is persisted in PostgreSQL and cached in Redis.
View citizen report history	`/reports/user`	The signed-in citizen can list reports associated with the current account.
Admin review and status update	Web dashboard and protected report routes	Administrators can list reports, inspect evidence, and update report status values via the gateway.
Live tracking broadcast	Socket.IO: `send-location`	Broadcasts valid coordinate payloads to connected clients; returns an error event for invalid payloads.

5.3 Analysis of Client Behavior

The mobile application is the main field interface and is implemented using Expo React Native [10, 11]. It offers citizen sign-in, view profile, report creation with evidence selection, see report history. Relief features are meant to read and map report information, not to create new reports, which fits with the planned separation of responsibilities between citizen reporting and relief awareness.

The online dashboard is developed using Next.js [12] and assessed as an administrative workspace. Key functions include reviewing reports, inspecting evidence and providing status updates. It also loads the current profile and denies access to the dashboard if a user fails role checks.

One of the weaknesses discovered is the data contract consistency in the client layer . The mobile code has logic for normalizing location fields such as `lat`/`lng` and `latitude`/`longitude`. This enhances tolerance during development but it shows that API contract should be locked down before wider use with consistent DTO validation and a single naming convention for coordinates and address fields.

5.4 Storage and Data Consistency

Evidence files are kept in Cloudinary while report data and evidence metadata are stored in PostgreSQL. This ensures that the database remains focused on organized fields and does not have to store raw media as database blobs [9]. Evidence metadata is saved as JSON fields (URL, public ID, resource type), which is a fair choice for a prototype, when the structure of the evidence is likely to change [8].

Redis cache is used sparingly. Create and update operations write a report cache entry . The report cache is emptied and refreshed when a report changes or is removed [21]. This method avoids multiple reads per report lookup but is not optimized for large list searches. If VietFlood scales to larger map queries, provincial filtering or huge result sets, caching and indexing should be extended beyond single report keys, and pagination should be enforced across list endpoints.

5.5 Access Control and Security Concerns

Route protection is implemented at the API gateway level with JWT authentication and role guards. Password is stored using bcrypt hashing [24]. Identity and role claims are carried in JWT access tokens that are utilized for authorization choices [6]. Refresh tokens

are issued with a different secret and expiration policy, and are meant to assist session control and logout.

There are still several security issues with prototypes. Test refresh token rotation and revocation with concurrent refresh token requests and replay attempts. Currently, Socket.IO tracking validates coordinate ranges, but the tracking channel should be coupled to verified IDs and a clearer authorization model. Evidence upload should also have stronger constraints on file size, kind and count to minimize abuse and unexpected storage cost.

5.6 Deployment Considerations

To deploy locally we execute the API gateway, services, RabbitMQ and Redis together in a repeatable environment with Docker Compose [13]. The prototype deployment workflow creates Docker images and deploys them to Azure App Service using GitHub Actions [14, 15]. Runtime configuration Environment variables for database connectivity, RabbitMQ, Redis, JWT secrets, refresh token secret, Cloudinary credentials, and admin seed settings.

The major operational gap is observability. The shared logger provides a standard logging interface, but a more robust implementation would additionally involve explicit health endpoints, service level metrics and alerting. These are helpful for spotting faults like upload evidence failures, queue issues or service unavailability without manual inspection.

5.7 Conclusions

The assessment indicates that VietFlood fits the core needs of the prototype. The app allows citizens to submit reports including location and evidence; the backend maintains reports and evidence metadata; administrators can check reports and alter status; and relief users may access report information with location context. The key gaps are hardening tasks required for genuine deployment: a reliable API contract, systematic tests under bad network conditions, better upload limits, tighter tracking authorization, and monitoring fit for unattended operation. In the context of the thesis, VietFlood shows that a microservice-based design with NestJS microservice support [19] may provide a suitable framework for multi-platform flood scenario reporting.

Chapter 6
Conclusion and Future Works

6.1 Final Summary

VietFlood was designed as a working prototype to translate ad-hoc flood observations into structured reports that can be searched, evaluated and updated. It's about reporting and coordination help, not about flood prediction. A citizen can file a report from a phone with location, description and optional photo evidence and that report is available for follow up instead of disappearing in chat history.

The supplied system consists of a NestJS backend divided into an API gateway, an authentication service and a reporting service, an Expo React Native mobile application for the citizens and relief flows and a Next.js dashboard for the administrators [18, 10, 12]. Reports are persisted in PostgreSQL using TypeORM, evidence is uploaded to Cloudinary and saved as metadata, and report lookups can be sped-up with Redis caching [7, 9, 21]. The gateway performs JWT based role checks for the segregation of citizen reporting, relief browsing and administrator review [6]. The RabbitMQ message patterns are used for the service-to-service work, so the HTTP concerns remain in the gateway and the business logic in internal services [20, 19]. This separation follows conventional microservice design recommendations, but is still small enough to be deployed as a prototype [3, 4, 5]

The prototype shows the complete cycle of reporting: report creation, archiving with evidentiary meta data, listing and retrieval using secure endpoints and change of status when approved by an administrator. In this version the relief role is deliberately read-focused. That decision maintains status updates in administrator control, since status is the primary trust signal that is shown back to other users.

In Chapter 5 the evaluation confirms the functional coverage by means of scenario walkthroughs and examination of the offered services and clients. VietFlood needs to be validated under real flood conditions with inadequate connectivity and more traffic before it can be considered for operational use.

6.2 Recommended Next Steps

The following enhancements should be based on genuine usage input on the reporting flow. If users are unable to quickly submit the minimum information (location, short description, and one photo), the system will not be useful in stressed settings.

From this prototype several concrete engineering tasks arise:

- **Offline-first reporting:** Local drafts, queued uploads, retry logic and clear client states for weak networks. Control abuse and storage cost by compressing evidence on-device and enforcing server-side restrictions.

- **Data quality and de-duplication:** make DTO naming and validation more stable and add duplicate identification based on time, distance and visual resemblance to reduce review workload.

- **Improved review workflow:** add clustering views, increase filters, support lightweight triage notes, so that administrators can evaluate bursts of incoming reports.

- **Tracking hardening:** bind Socket.IO channels to authenticated identities and enforce permission rules in line with the planned operational policy.
- **Operations and security:** add health checks, monitoring and failure alarms, test refresh token rotation under concurrency, and improve deployment methods for repeatability and recovery [16, 17, 13].

In a longer project schedule VietFlood can also be expanded beyond reporting: integration with governmental warning channels, evidence verification collaborations and map-oriented aggregation for public information. The handling of those extensions should be done with care, as public facing status and location data brings privacy and safety problems that are out of the scope of this thesis.

Bibliography

[1] M. F. Goodchild and J. A. Glennon, "Crowdsourcing geographic information for disaster response: a research frontier," *International Journal of Digital Earth*, vol. 3, no. 3, pp. 231–241, 2010.

[2] M. Imran, C. Castillo, F. Diaz, and S. Vieweg, "Processing social media messages in mass emergency: A survey," *ACM Computing Surveys*, vol. 47, no. 4, pp. 67:1–67:38, 2015.

[3] M. Fowler and J. Lewis, "Microservices: A definition of this new architectural term." `https://martinfowler.com/articles/microservices.html`, 2014. Accessed: 2026-05-17.

[4] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, pp. 195–216, Springer International Publishing, 2017.

[5] J. Thönes, "Microservices," *IEEE Software*, vol. 32, no. 1, p. 116, 2015.

[6] M. B. Jones, J. Bradley, and N. Sakimura, "Json web token (jwt)," RFC 7519, Internet Engineering Task Force, 2015. Accessed: 2026-05-17.

[7] TypeORM, "Typeorm documentation: Getting started." `https://typeorm.io/docs/getting-started/`, 2026. Accessed: 2026-05-17.

[8] PostgreSQL Global Development Group, "Postgresql documentation: Json types." `https://www.postgresql.org/docs/current/datatype-json.html`, 2026. Accessed: 2026-05-17.

[9] Cloudinary, "Upload api reference." `https://cloudinary.com/documentation/image_upload_api_reference`, 2026. Accessed: 2026-05-17.

[10] Expo, "Expo documentation." `https://docs.expo.dev/`, 2026. Accessed: 2026-05-17.

[11] Meta Open Source, "React native documentation: Getting started." `https://reactnative.dev/docs/getting-started`, 2026. Accessed: 2026-05-17.

[12] Vercel, "Next.js documentation." `https://nextjs.org/docs`, 2026. Accessed: 2026-05-17.

[13] Docker, "Docker compose documentation." `https://docs.docker.com/compose/`, 2026. Accessed: 2026-05-17.

[14] GitHub, "Github actions documentation." `https://docs.github.com/en/actions/`, 2026. Accessed: 2026-05-17.

[15] Microsoft, "Continuous deployment with custom containers in azure app service." `https://learn.microsoft.com/en-us/azure/app-service/deploy-ci-cd-custom-container`, 2026. Accessed: 2026-05-17.

[16] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.

[17] G. Kim, P. Debois, J. Willis, and J. Humble, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, 2016.

[18] NestJS, "Nestjs documentation." `https://docs.nestjs.com/`, 2026. Accessed: 2026-05-17.

[19] NestJS, "Nestjs microservices documentation." `https://docs.nestjs.com/microservices/basics`, 2026. Accessed: 2026-05-17.

[20] RabbitMQ, "Consumer acknowledgements and publisher confirms." `https://www.rabbitmq.com/docs/3.13/confirms`, 2026. Accessed: 2026-05-17.

[21] Redis, "Redis documentation." `https://redis.io/docs/latest/`, 2026. Accessed: 2026-05-17.

[22] Vietnam Open API, "Vietnam provinces open api." `https://provinces.open-api.vn/`, 2026. Accessed: 2026-05-17.

[23] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley Professional, 4 ed., 2021.

[24] N. Provos and D. Mazières, "A future-adaptable password scheme," in *1999 USENIX Annual Technical Conference*, (Monterey, California), USENIX Association, June 1999. Accessed: 2026-05-17.

[25] Socket.IO, "Socket.io documentation." `https://socket.io/docs/v4/`, 2026. Accessed: 2026-05-17.

Appendix A
API Reference and Sample Payloads

This appendix summarizes the VietFlood API gateway endpoints and provides representative JSON responses. Token strings, user identifiers, and media URLs are masked with placeholders.

A.1 Endpoint Inventory

Table A.1: REST endpoints exposed by the VietFlood API gateway.

Method	Endpoint	Access	Description
Authentication			
POST	`/auth/register`	Public	Create a new user account.
POST	`/auth/sign_in`	Public	Authenticate and obtain access and refresh tokens.
POST	`/auth/refresh_token`	Refresh token	Issue a new access token and rotate the refresh token.
POST	`/auth/logout`	Public	Logout and revoke refresh token (if provided).
POST	`/auth/refresh`	Public	Refresh helper endpoint used by the gateway (payload passthrough).
GET	`/auth/profile`	JWT	Fetch the current user profile.
PUT	`/auth/update`	JWT	Update the current user's profile.
PUT	`/auth/update/user/:id`	JWT + Role (admin/relief)	Update a user profile by user id.
DELETE	`/auth/delete/:id`	JWT + Role (admin)	Delete a user by user id.
GET	`/auth/all`	JWT + Role (admin/relief)	List all users.
GET	`/auth/relief/users`	JWT + Role (admin/relief)	List users (alias of `/auth/all`).
GET	`/auth/relief/users/:id`	JWT + Role (admin/relief)	Fetch a user by user id.
Reports			
POST	`/reports/create`	JWT	Create a new flood report (supports evidence file upload).
GET	`/reports`	JWT + Role (admin/relief)	Fetch all reports for admin/relief review.
GET	`/reports/user`	JWT + Role (citizen)	Fetch reports created by the current citizen user.
GET	`/reports/:id`	JWT + Role (admin/relief)	Fetch reports for a specific user id.
PUT	`/reports/update/:id`	JWT	Update a report owned by the current user (supports evidence upload).
PUT	`/reports/update/:id/admin/:userId`	JWT + Role (admin)	Update a report for a specific user id (admin workflow).
PUT	`/reports/relief/:id/user/:userId`	JWT + Role (admin/relief)	Update a report for a specific user id (relief workflow).
DELETE	`/reports/:id`	JWT	Delete a report owned by the current user.
DELETE	`/reports/update/:id/admin/:userId`	JWT + Role (admin)	Delete a report for a specific user id (admin workflow).
DELETE	`/reports/relief/:id/user/:userId`	JWT + Role (admin/relief)	Delete a report for a specific user id (relief workflow).

A.2 Authentication: Sign In

Endpoint: `POST /auth/sign_in`

Response body:

```
{
  "accessToken": "<JWT_ACCESS_TOKEN>",
```

```
  "refresh_token": "<JWT_REFRESH_TOKEN>"
}
```

Listing A.1: Sign In Response

A.3 Authentication: Token Refresh

Endpoint: `POST /auth/refresh_token`

Response body:

```
{
  "access_token": "<JWT_ACCESS_TOKEN>",
  "refresh_token": "<JWT_REFRESH_TOKEN>"
}
```

Listing A.2: Refresh Token Response

A.4 User: Profile Payload

Endpoint: `GET /auth/profile`

Response body:

```
{
  "id": 7,
  "email": "<user_email>",
  "username": "<username>",
  "phone": "<phone_number>",
  "role": "citizen",
  "first_name": "<first_name>",
  "middle_name": null,
  "last_name": "<last_name>",
  "date_of_birth": "2000-01-01",
  "address_line": "<address_line>",
  "ward": "<ward>",
  "province": "<province>",
  "created_at": "2026-05-17T00:00:00.000Z",
  "updated_at": "2026-05-17T00:00:00.000Z"
}
```

Listing A.3: User Profile Response

A.5 Reports: Create Payload

Endpoint: `POST /reports/create`

Response body:

```
{
  "success": true,
  "message": "Create report successfully",
  "reportId": 41,
  "category": ["flood"],
  "evidences": [
    {
      "url": "<CLOUDINARY_URL>",
      "publicId": "<CLOUDINARY_PUBLIC_ID>",
      "resourceType": "image"
    }
  ],
  "images": [
    "<CLOUDINARY_URL>"
  ]
}
```

Listing A.4: Create Report Response

Appendix B
User Interface Snapshots

Key user interface screens are shown here. These images support the main report without repeating the implementation details.

B.1 Mobile and Dashboard Screens

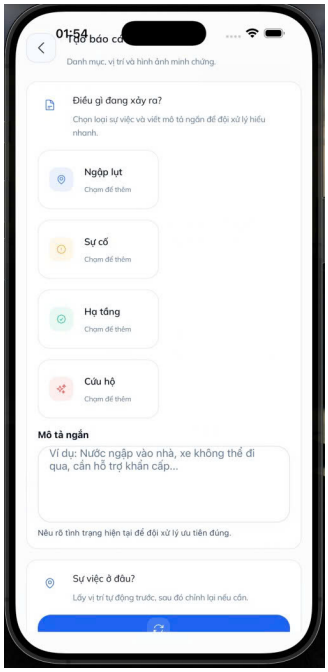


Figure B.1: Mobile report creation screen (citizen workflow).

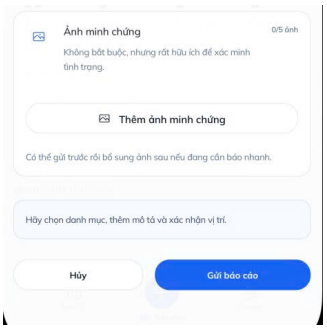


Figure B.2: Mobile evidence selection and upload screen.

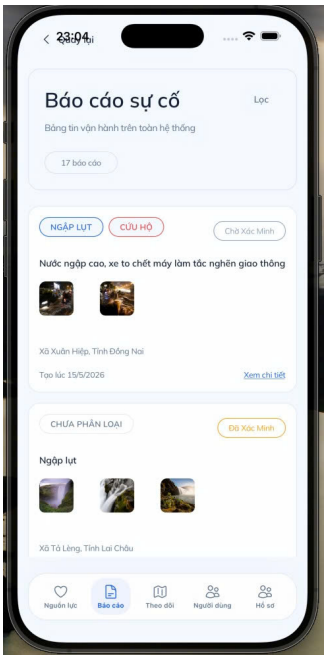


Figure B.3: Relief report list screen for browsing incoming records.

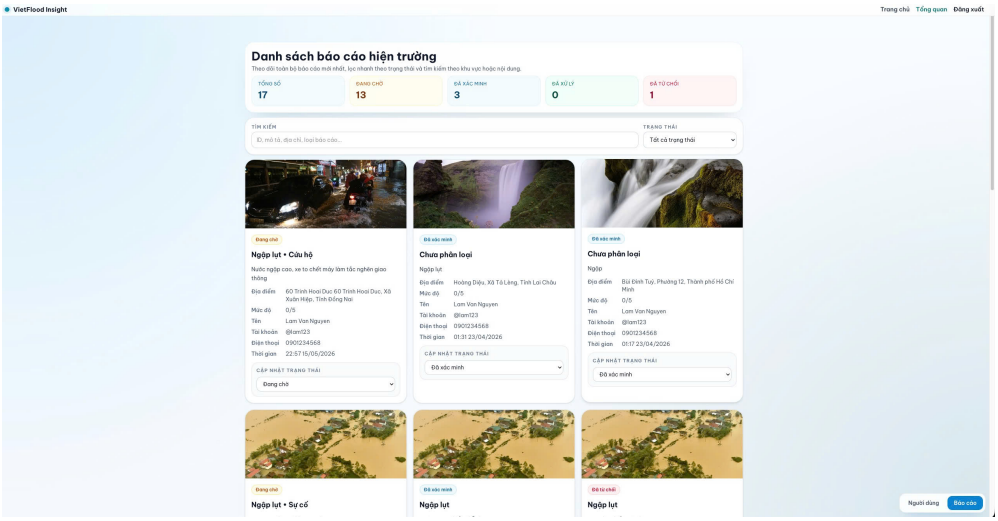


Figure B.4: Administrator dashboard overview with report list and filters.

Appendix C
Prototype Validation Checklist

This appendix lists system-level test cases used to validate the VietFlood prototype. The cases focus on end-to-end behavior through the API gateway, including authentication, role checks, report workflow, and evidence handling.

C.1 Auth and User Scenarios

Table C.1: Authentication and user management validation scenarios.

ID	Feature	Input / Steps (summary)	Expected Result
AUTH-01	Sign in success (`POST /auth/sign_in`)	Provide valid `username` and `password`.	Response contains `accessToken` and `refresh_token`.
AUTH-02	Sign in failure	Provide valid username with incorrect password.	Request rejected with an authentication error (no token returned).
AUTH-03	Access protected route without JWT	Call `GET /auth/profile` without `Authorization: Bearer ....`	Request rejected.
AUTH-04	Refresh token success (`POST /auth/refresh_token`)	Send `refresh` value in the body containing a valid refresh token.	Response contains `access_token` and a rotated `refresh_token`.
AUTH-05	Role enforcement	Call admin/relief-only routes (for example `GET /reports`) using a citizen JWT.	Request rejected due to insufficient role.
AUTH-06	Admin user listing	Call `GET /auth/all` with an admin JWT.	Returns list of users with profile fields.

C.2 Report and Evidence Scenarios

Table C.2: Report workflow and evidence handling validation scenarios.

ID	Feature	Input / Steps (summary)	Expected Result
REP-01	Create report with evidence (`POST /reports/create`)	Submit required fields (`description`, `province`, `ward`, `addressLine`) with `category[]` and 1–3 image files.	Response indicates success and returns `reportId`, `evidences` metadata, and `images[]` URLs.
REP-02	Create report without evidence	Submit report payload with no files.	Report created; `evidences` and `images` are empty arrays.
REP-03	Citizen fetches own reports (`GET /reports/user`)	Call as citizen with valid JWT.	Returns list of reports created by the current user.
REP-04	Admin/relief fetches all reports (`GET /reports`)	Call as admin or relief user.	Returns list of report records for review.
REP-05	Update report by owner (`PUT /reports/update/:id`)	Update an existing report id owned by the current user; optionally attach additional evidence files.	Returns success message and updated evidence list and image URLs.
REP-06	Delete report by owner (`DELETE /reports/:id`)	Delete an existing report id owned by the current user.	Returns success confirmation; subsequent fetch does not include the deleted report.

C.3 Tracking Scenarios (Socket.IO)

Table C.3: Socket.IO live tracking validation scenarios.

ID	Event	Input / Steps (summary)	Expected Result
TRK-01	Broadcast valid location (`send-location`)	Connect a client and emit `send-location` with valid latitude/longitude ranges.	Server broadcasts `receive-location` containing the sender id and payload.
TRK-02	Reject invalid payload	Emit `send-location` with missing or out-of-range coordinates.	Client receives `location-error` and location is not broadcast.
TRK-03	Disconnect notification	Disconnect a client socket.	Server broadcasts `user-disconnected` with the socket id.

Appendix D
Runtime Configuration Summary

This chapter summarizes the environment variables used to deploy VietFlood with Docker Compose. Values shown here are variable names and example formats only. Secret values are not included.

Table D.1: Environment variable groups used for VietFlood deployment with Docker Compose.

Group	Example Variables	Notes
Gateway	`API_GATEWAY_PORT`, `CORS_ORIGINS`	HTTP port and allowed web origins for the gateway service.
Runtime	`NODE_ENV`	Runtime mode used by the Node.js applications (for example, `production`).
Database	`DATABASE_URL`	PostgreSQL connection string for the deployment database.
Message broker	`RABBITMQ_DEFAULT_USER`, `RABBITMQ_DEFAULT_PASS`, `RABBITMQ_URL`	RabbitMQ credentials and connection URL used by the backend services.
Cache	`REDIS_HOST`, `REDIS_PORT`, `REDIS_PASSWORD`, `REDIS_DB`	Redis connection details used for caching (host, port, password, database index).
Token signing	`JWT_SECRET`, `REFRESH_SECRET`	Secrets used to sign access tokens and refresh tokens.
Admin seed	`ADMIN_EMAIL`, `ADMIN_PASSWORD`	Initial administrator credentials used during seed/startup logic.
Cloudinary	`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	Cloudinary credentials used for evidence upload and deletion.

D.1 Operational Notes

- Use masked values for secrets in the thesis and replace them with actual values in deployment.
- Store credentials in CI/CD secrets or environment configuration, not in source control.
- Change default RabbitMQ credentials before production use.
- Set JWT secrets as cryptographically secure strings.